Activity: Group REST API Enhancement and Multi- Client Implementation

API Documentation

This API is designed for a simple user authentication system that supports both student and faculty roles. It allows users to register and log in using JSON requests. The API processes the input data, validates it, and stores user information in a MySQL database named adet_api_db. During registration, the system requires a username, email, password, and role. It checks if all required fields are provided, ensures the username, email, and password follow validation rules, and confirms that the role is either “student” or “faculty.” It also checks for duplicate accounts by verifying if the username or email already exists in the database. If the input is valid, the password is securely hashed before being saved. During login, the API verifies the username and password. If the credentials match, it returns a successful response along with the user’s role. This allows different client applications to handle users based on their role. The API uses prepared statements to protect against SQL injection and uses password hashing for secure authentication.

Base URL:
http://localhost/rest-api-project/server/server.php

Register User:

- URL: /server.php
- Method: POST
- Description: Registers a new user into the system database.

Request Parameters

Parameter	Type	Required	Description
username	string	Yes	User’s username (minimum 3 characters)
password	string	Yes	User’s password (minimum 4 characters)
email	string	Yes	User’s email address
roles	string	Yes	User role (student or faculty)
action	string	Yes	Must be “register”

API Logic Flow

The API follows a step-by-step process to ensure correct data handling and security.

1. Request Validation
   - The API only accepts POST requests.
   - If another request method is used, it is rejected.

2. Database Connection

   - The API connects to MySQL database adet_api_db using port 3306.
   - If the connection fails, the system returns a database error message.

3. Action Check

   - The API reads the action field from the request:
     1. "register" → user registration
     2. "login" → user authentication

   - If no action is provided, the request is rejected.

4. Input Validation

   - For Registration:

The API checks if the following fields exist:

1. Username
2. Password
3. Email
4. role

It also validates:
- Username must be at least 3 characters
- Password must be at least 4 characters
- Email must be a valid format
- Role must be either “student” or “faculty”

5. Duplicate User Check (Registration Only)

   - The system checks the database for existing users using:

1. Username
2. email

- If a match is found, registration is blocked.

6. Password Security
- The password is encrypted using:
    1. password_hash() during registration
    2. password_verify() during login

7. Data Processing

Registration:
- If validation passes:
1. User data is inserted into the users table
2. Stored fields: username, email, hashed password, role

Login:
- The API searches for the username
- If found, password is verified using password_verify()

8. Response Generation

- The API returns JSON responses:
    1. success → operation successful
    2. error → system or validation error
    3. invalid → incorrect input format
    4. exists → user already registered

- Login success also returns the user's role.

9. End Process

- After sending the response, the API ends the request.

Request Body:

```
{
  "action": "register",
  "username": "user00",
  "email": "abc29100-8225-39498@bicol-u.edu.ph",
  "password": "qweasdzxc",
  "roles": "student"
}
```

Success Response

Response Headers 9 Cookies Results Docs

```
{
  "status": "success",
  "message": "Registered successfully",
  "role": "student"
}
```

Invalid Input

1. Username too short

Response Headers 9 Cookies Results Docs

```
{
  "status": "invalid_input",
  "message": "Username must be at least 3 characters"
}
```

2. Password too short

Response Headers 9 Cookies Results Docs

```
{
  "status": "invalid_input",
  "message": "Password must be at least 4 characters"
}
```

Allowed Methods

POST – used for registration
OPTIONS – used for CORS preflight requests

Security Features
Password hashing using password_hash()
Password verification using password_verify()
SQL Injection protection using prepared statements ($conn->prepare())

Database Table

	Table	Action	Rows	Type	Collation	Size	Overhead
☐	users	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	16.0 KiB	-
	1 table	Sum	6	InnoDB	utf8mb4_general_ci	16.0 KiB	0 B

←T→				id	username	password	email	roles
☐	Edit	Copy	Delete	18	gailanne	$2y$10$VR985gSvDR7TLQaXEg8LxuMCtiv4w0rMlnDQawtJ4tx...	gail2@bicol-u.edu.ph	student
☐	Edit	Copy	Delete	19	hannahncl	$2y$10$8P6wrk5Ymm4xh4n3688bZui/y22pJIDMm1eHUot4Bdp...	hannahnc@bicol-u.edu.ph	student
☐	Edit	Copy	Delete	20	jamesr	$2y$10$JQ8ecf7uR78/KkeaP.qtyOPIDEpVt7Fa71DPmRyLqDm...	jamesrmos@bicol-u.edu.ph	student
☐	Edit	Copy	Delete	21	lukef	$2y$10$MQ6UPxFNbJejCX3jtkIZ/.MduNKcw1iV9L4NoDq/JCX...	arsyn@bicol-u.edu.ph	student
☐	Edit	Copy	Delete	22	jorgeb	$2y$10$RpEsITr/tRFqxw.n3T7iEu.gVH69WwIP5OmM.PVjUX....	tyson@bicol-u.edu.ph	student
☐	Edit	Copy	Delete	23	owhiel	$2y$10$.aPYNMADYv1YrWWnI8bg.eaGDpCIGZlZeQBhTZQUq6G...	owhie@bicol-u.edu.ph	faculty

Testing Results

Test Case 1: Successful Registration

Test Case 2: Duplication

Test 3: Password Error Handling